

Introduction to Java EE

Developers today increasingly recognize the need for distributed, transactional, and portable applications that leverage the speed, security, and reliability of server-side technology. Enterprise applications provide the business logic for an enterprise. They are centrally managed and often interact with other enterprise software. In the world of information technology, enterprise applications must be designed, built, and produced for less money, with greater speed, and with fewer resources.

With the Java Platform, Enterprise Edition (Java EE), development of Java enterprise applications has never been easier or faster. The aim of the Java EE platform is to provide developers with a powerful set of APIs while shortening development time, reducing application complexity, and improving application performance.

The Java EE platform is developed through the Java Community Process (JCP), which is responsible for all Java technologies. Expert groups composed of interested parties have created Java Specification Requests (JSRs) to define the various Java EE technologies. The work of the Java Community under the JCP program helps to ensure Java technology's standards of stability and cross-platform compatibility.

The Java EE platform uses a simplified programming model. XML deployment descriptors are optional. Instead, a developer can simply enter the information as an annotation directly into a Java source file, and the Java EE server will configure the component at deployment and runtime. These annotations are generally used to embed in a program data that would otherwise be furnished in a deployment descriptor. With annotations, you put the specification information in your code next to the program element affected.

In the Java EE platform, dependency injection can be applied to all resources a component needs, effectively hiding the creation and lookup of resources from application code. Dependency injection can be used in Enterprise JavaBeans (EJB) containers, web containers, and application clients. Dependency injection allows the Java EE container to automatically insert references to other required components or resources, using annotations.

Introduction to JavaBeans

Bean is a reusable software component model based on sun java bean specification. One of the most important topics in software component model is reusability. Writing self-contained software under the moto of "Developed them once, run and reuse them everywhere" is most required and appreciated. So reusability is added into the Java programming language with the help of JavaBeans.

What are JavaBeans?

JavaBeans are introduced in 1996 by Sun Microsystem and defined as **"A JavaBean is reusable, platform independent component that can be manipulated visually in a builder tool."**

In computing, based on the Java Platform, JavaBeans are classes that encapsulate many objects into a single object (the bean). Builder tool enables you to create and use beans for application development purpose. In simple words JavaBean is nothing but a Java class. When these JavaBeans are used in other applications, the internal working of such components are hidden from the application developer.

Example:

All Swing and AWT classes are JavaBeans. GUI components are ideal JavaBeans.

Components of JavaBeans

The classes that contained definition of beans is known as components of JavaBeans. These classes follows certain design conventions. It includes properties, events, methods and persistence. There are two types of components, GUI based and non GUI based. For instance JButton is example of a component not a class.

Properties (data members): Property is a named attribute of a bean, it includes color, label, font, font size, display size. It determines appearance, behavior and state of a bean.

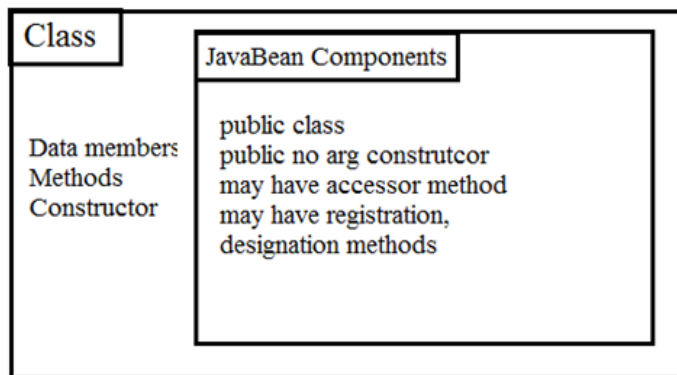
Methods: Methods in JavaBeans are same as normal Java methods in a class. It doesn't follow any specific naming conventions. All properties should have accessor and getter methods.

Events: Events in JavaBeans are same as SWING/AWT event handling.

Persistence: Serializable interface enables JavaBean to store its state. JavaBean has no argument constructor.

JavaBeans Properties

JavaBean property can be access by the user of the object, it can be read, write, read only or write only. We can access these JavaBeans properties with the help of `getPropertyName()` method also known as getter or accessor and `setPropertyname()` method known as setter written in



implementation class of bean.

getPropertyName(): For example, if property name is Title, your method name would be getTitle().

setPropertyName(): For example, if property name is Title, your method name would be setTitle().

Example of JavaBeans

Before going to write a JavaBean, here are some basic rules. A JavaBean should be public, should has no argument default constructor and should implement serializable interface. Keep these basic rules in mind before writing a JavaBean.

Here is a simple example of JavaBean.

```
public class BankAccount implements java.io.Serializable {
    private String accountNumber = null;
    private String totalAmount = null;
    private String accountHolderName = null;
    private int accountHolderAge = 0;
    private String accountHolderAddress = null;

    public String getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getTotalAmount() {
        return totalAmount;
    }

    public void setTotalAmount(String totalAmount) {
        this.totalAmount = totalAmount;
    }

    public String getAccountHolderName() {
        return accountHolderName;
    }

    public void setAccountHolderName(String accountHolderName) {
        this.accountHolderName = accountHolderName;
    }
}
```

```

    }

    public int getAccountHolderAge() {
        return accountHolderAge;
    }

    public void setAccountHolderAge(int accountHolderAge) {
        this.accountHolderAge = accountHolderAge;
    }

    public String getAccountHolderAddress() {
        return accountHolderAddress;
    }

    public void setAccountHolderAddress(String accountHolderAddress) {
        this.accountHolderAddress = accountHolderAddress;
    }
}

```

Advantages of JavaBeans

Following are some advantages of JavaBeans:

- Reusability in different environments.
- Used to create applet, servlet, application or other components.
- JavaBeans are dynamic, can be customized.
- Can be deployed in network systems

Writing Simple JavaBean Example

compose.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Compose Message</title>
</head>
<body>
    <form action="sender.jsp" method="post">
        Email Address: <input type="email" name="email" /><br /> Message<br />
        <textarea name="message"></textarea>
        <input type="submit" value="SEND" name="send" />
    </form>

```

```

        </form>
    </body>
</html>

```

Message.java

package mybean;

import java.io.Serializable;

public class Message **implements** Serializable {

private static final long *serialVersionUID* = 1L;

private String email;

private String message;

public Message(){
 }

public String getEmail() {
 return email;
 }

public void setEmail(String email) {
 this.email = email;
 }

public String getMessage() {
 return message;
 }

public void setMessage(String message) {
 this.message = message;
 }

}

sender.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Sender</title>
</head>

```

```
<body>
  <jsp:useBean id="msg" scope="request" class="mybean.Message" />
  <jsp:setProperty property="*" name="msg" />
  <jsp:forward page="display.jsp"></jsp:forward>
</body>
</html>
```

display.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Display Message</title>
</head>
<body>
  <jsp:useBean id="msg" scope="request" class="mybean.Message" />
  <label>Email Address: </label><jsp:getProperty property="email"
    name="msg" />
  <hr />
  <label>Message: </label><jsp:getProperty property="message" name="msg" />
</body>
</html>
```

What is EJB

EJB is an acronym for *enterprise java bean*. It is a specification provided by Sun Microsystems to develop secured, robust and scalable distributed applications.

To run EJB application, you need an *application server* (EJB Container) such as Jboss, Glassfish, Weblogic, Websphere etc. It performs:

- a. life cycle management,
- b. security,
- c. transaction management, and
- d. Object pooling.

EJB application is deployed on the server, so it is called server side component also.

EJB is like COM (*Component Object Model*) provided by Microsoft. But, it is different from Java Bean, RMI and Web Services.

When use Enterprise Java Bean?

1. **Application needs Remote Access.** In other words, it is distributed.

2. **Application needs to be scalable.** EJB applications supports load balancing, clustering and fail-over.
3. **Application needs encapsulated business logic.** EJB application is separated from presentation and persistent layer.

Types of Enterprise Java Bean

There are 3 types of enterprise bean in java.

Session Bean

Session bean contains business logic that can be invoked by local, remote or webservice client.

Message Driven Bean

Like Session Bean, it contains the business logic but it is invoked by passing message.

Entity Bean

It encapsulates the state that can be persisted in the database. It is deprecated. Now, it is replaced with JPA (Java Persistent API).

Difference between RMI and EJB

Both RMI and EJB, provides services to access an object running in another JVM (known as remote object) from another JVM. The differences between RMI and EJB are given below:

RMI	EJB
In RMI, middleware services such as security, transaction management, object pooling etc. need to be done by the java programmer.	In EJB, middleware services are provided by EJB Container automatically.
RMI is not a server-side component. It is not required to be deployed on the server.	EJB is a server-side component, it is required to be deployed on the server.
RMI is built on the top of socket programming.	EJB technology is built on the top of RMI.

EJB and Webservice

In EJB, bean component and bean client both must be written in java language.

If bean client need to be written in other language such as **.net, php** etc, we need to go with **webservices** (SOAP or REST). So EJB with web service will be better option.

Disadvantages of EJB

1. Requires application server
2. Requires only java client. For other language client, you need to go for webservice.
3. Complex to understand and develop ejb applications.

Session Bean

Session bean encapsulates business logic only, it can be invoked by local, remote and webservice client.

It can be used for calculations, database access etc.

The life cycle of session bean is maintained by the application server (EJB Container).

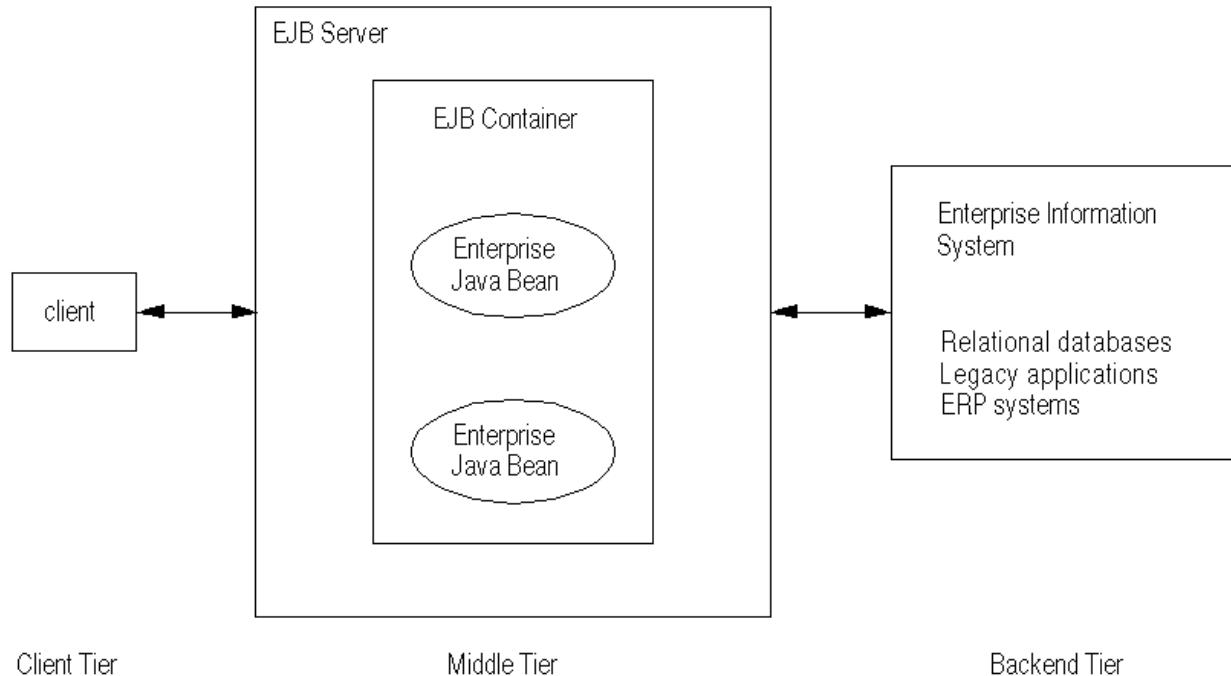
Types of Session Bean

There are 3 types of session bean.

- 1) Stateless Session Bean:** It doesn't maintain state of a client between multiple method calls.
- 2) Stateful Session Bean:** It maintains state of a client across multiple requests.
- 3) Singleton Session Bean:** One instance per application, it is shared between clients and supports concurrent access.

Components of EJB Architecture

- EJB server(s)
- EJB container(s)
- Enterprise Beans
- EJB clients
- other auxillary systems (e.g. Java Naming and Directory Interface (JNDI) server, Java Transaction Service (JTS), ...).



Roles in development of the application based on distributed components

The complexity and diversity of tasks needed to write a robust distributed application calls for separation of responsibilities to multiple domain experts.

The EJB specification assigns specific roles for project participants charged with enterprise application development utilizing EJBs:

EJB server provider

- provides an application framework in which to run the EJB containers
- implements and provide access to a JNDI-compatible naming service
- implements a transaction service compatible with CORBA-OTS (Object Transaction Service)

EJB container provider

- supplies runtime classes that provide the required services to the EJB Bean instances
- provides deployment tools to
 - generate stubs and skeletons classes to provide access to the EJB instance's Home object and EJBObject (the proxy to the bean itself)
 - install an EJB Bean with its supporting classes on an EJB server
 - register the reference to the Home object in a JNDI-accessible namespace

- generate EJBObject implementation that can provide correct proxy services for the bean class

EJB provider

- responsible for coding the business logic into server side components (beans)
- should also have knowledge of the EJB specification
- describes transactional needs of the various methods in the EJB class to the EJB deployer

EJB deployer

- have an in-depth understanding of the characteristics of the particular run-time server environment such as database type, its location, etc.
- installs the bean and its supporting classes on the EJB server and registers its home interface with JNDI
- may not be a Java developer nor understand the business rules that an EJB class implements

The deployer gets the EJB class requirements from the EJB developer such as transactional needs, names and descriptions of the required environment properties and so forth. The deployer is responsible for making these properties, along with their correct runtime values available to the EJB class at runtime.

Application assembler

- writes the client applications using pre-built EJB components, configuring and customizing them for the assembled application

EJB Server

- provides the execution environment for EJB containers
- provides the system services (multiprocessing, load-balancing, device access) for EJB containers
- effectively manages system resources for the container (pools and recycle network connections, processes, threads, memory, database connections, ...)
- solves the concurrent access to the bean (commonly, serialization of bean method's accesses is used)
- provides implementations of naming server (JNDI), transaction services (acts as transaction processing monitor) and optionally other (e.g. security) services

EJB Containers

- act as the interface between an Enterprise Java Bean and low-level, platform-specific functionality that supports the Bean - the EJB Server

- EJB client never accesses a bean directly. Any bean access is done through container-generated class which in turn invokes the bean's methods
- by intercepting invocations of bean's methods, the container may provide system-level services to beans (transaction, security, control of concurrent access ...)
- the transaction, persistence and security management is based on the declarative approach (i.e. the actual behavior with respect to those services is specified in the deployment descriptor and thus is not hard-coded in the bean's class code)
- container manages a lifecycle of beans to utilize server resources (memory, threads) effectively. It may passivate the instance of a bean (i.e. flush its state to a database and deactivate it) and activate it (i.e. create a new instance and restore its original state from the database) when the client invokes the bean's method later.
- container makes the required services (e.g. transaction service) available to beans through standard interfaces defined in the EJB specification

The container tightly interacts with classes generated by the deployment tool for the deployed EJB Bean to manage enterprise Bean's persistence, transactional properties and security.

Note: The EJB Server may host one or more EJB Containers, possibly each of them configured differently.

EJB Clients

- Clients interact with Enterprise Beans instances through proxy objects (generated by tools provided by container provider). Commonly, the communication is based on IIOP protocol and proxy objects are regular CORBA stubs
- the presence of the container wrapping the bean is completely hidden to the client
- the passivation and activation of bean instances is completely transparent to the client
- The reference to the bean is acquired from the factory (called "home interface") which is specific for each bean type. The factory is capable for creating new bean instances and finding the existing ones (based on some searching criteria). The reference to the factory itself is obtained using the Java Naming and Directory (JNDI) interface
- due to selection IIOP as the communication protocol, the bean may also integrate with RMI/IIOP application or even the client written in any programming language supported by CORBA

Parts of the EJB Bean

Home Interface allows client to create, remove and find existing bean instances

Remote Interface exposes the business logic of the bean to the client

EJB Class provides the actual implementation of the business logic (implements methods defined in the Remote interface). Also Implements the creation and finder methods declared by the Home interface.

Deployment Descriptor specifies the configurable properties of the bean, its transactional, security and persistent state management policy